

LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	Experiment No.:	TASK 3
Course Code:	23CM4121	Date:	30-08-2026
Student Name:	ROWTHU BHASKAR ABHIRAM	Roll No.:	A24126552114

1. TITLE

Interactive To-Do List Application Using JavaScript, DOM Manipulation, and Event Listeners

2. AIM

To develop an interactive To-Do List web application using JavaScript, DOM manipulation, and event listeners, which allows the user to add, complete, and delete tasks dynamically.

3. OBJECTIVE

- To provide a text box and an Add Task button for entering new tasks.
- To dynamically create a new task item and add it to a list when the Add Task button is clicked.
- To provide a Complete button for each task that visually marks the task as completed when clicked.
- To provide a Delete button for each task that removes the corresponding task from the webpage when clicked.
- To display an appropriate message when the task list is empty.
- To demonstrate DOM element selection, dynamic element creation, dynamic modification of the DOM, and event handling.

4. THEORY

DOM Element Selection: `document.getElementById()` is used to select existing elements such as the input box, Add Task button, task list, and the empty-state message, so that JavaScript can read their values or modify them.

Dynamic Element Creation: `document.createElement()` is used to create new elements such as the list item (li), the task text (span), and the Complete/Delete buttons entirely in JavaScript, at the moment a new task is added.

Dynamic Modification of the DOM: Properties such as `className` and `textContent` are used to set the CSS class and visible text of newly created elements, and `classList.toggle()` is used to add or remove the "completed" style on a task's text.

Event Listeners and Click Handling: `addEventListener("click", ...)` attaches functions to the Add Task, Complete, and Delete buttons so that they respond when clicked. A "keydown" listener on the input box also allows a task to be added by pressing Enter.

Adding and Removing Elements: `appendChild()` inserts newly created elements (the task item, its text, and its buttons) into the task list, while `element.remove()` deletes a task item from the DOM when its Delete button is clicked.

Interactive Webpage Development: Combining the above techniques allows the page to respond immediately to user actions without reloading, updating the empty-list message automatically as tasks are

added or removed.

5. ALGORITHM / PROCEDURE

- 1 Create the HTML page with a text input (id="taskInput"), an Add Task button (id="addBtn"), an empty-state message (id="emptyMessage"), and an empty unordered list (id="taskList").
- 2 Link an external CSS file to style the container, input box, buttons, task items, and the completed-task text.
- 3 In the JavaScript file, select the input box, Add Task button, task list, and empty message using document.getElementById().
- 4 Define an updateEmptyState() function that shows the empty message when the task list has no children, and hides it otherwise.
- 5 Define an addTask() function that reads and trims the value from the input box, and shows an alert if it is empty.
- 6 Inside addTask(), dynamically create a list item, a span for the task text, a button container, a Complete button, and a Delete button using document.createElement().
- 7 Attach a click event listener to the Complete button that toggles a "completed" CSS class on the task text.
- 8 Attach a click event listener to the Delete button that removes the task item from the DOM and calls updateEmptyState().
- 9 Append the buttons to the button container, and append the text and button container to the list item, then append the list item to the task list.
- 10 Clear the input box, refocus it, and call updateEmptyState() after a task is added.
- 11 Attach a click event listener to the Add Task button that calls addTask(), and a keydown listener on the input box that calls addTask() when the Enter key is pressed.
- 12 Save the HTML, CSS, and JavaScript files and open the HTML file in a browser to verify adding, completing, and deleting tasks, as well as the empty-list message.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List Application</title>
  <link rel="stylesheet" href="style3.css">
</head>
<body>

  <div class="container">
    <h1>My To-Do List</h1>

    <div class="input-group">
      <input type="text" id="taskInput" placeholder="Enter a task...">
      <button id="addBtn">Add Task</button>
    </div>

    <p id="emptyMessage">No tasks available.</p>
    <ul id="taskList"></ul>
  </div>

  <script src="script3.js"></script>
</body>
</html>
```

style3.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f6f8;
  padding: 30px;
  display: flex;
  justify-content: center;
}

.container {
  background-color: #ffffff;
  width: 480px;
  padding: 25px;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

h1 {
  text-align: center;
  color: #333333;
  margin-top: 0;
}

.input-group {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

#taskInput {
  flex: 1;
  padding: 10px 12px;
  font-size: 14px;
  border: 1px solid #ccc;
  border-radius: 4px;
  outline: none;
}

#taskInput:focus {
  border-color: #007bff;
}

#addBtn {
  padding: 10px 16px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
}
```

```
        cursor: pointer;
        font-weight: bold;
    }

    #addBtn:hover {
        background-color: #0056b3;
    }

    #taskList {
        list-style: none;
        padding: 0;
        margin: 0;
    }

    .task-item {
        display: flex;
        align-items: center;
        justify-content: space-between;
        background-color: #fafafa;
        padding: 10px 12px;
        margin-bottom: 8px;
        border: 1px solid #e0e0e0;
        border-radius: 4px;
    }

    .task-text {
        flex: 1;
        font-size: 15px;
        color: #333;
        word-break: break-word;
        margin-right: 10px;
    }

    /* Completed task style */
    .completed {
        text-decoration: line-through;
        color: #888888;
    }

    .btn-group {
        display: flex;
        gap: 6px;
    }

    .complete-btn {
        background-color: #28a745;
        color: white;
        border: none;
        padding: 6px 12px;
        border-radius: 4px;
        cursor: pointer;
    }

    .complete-btn:hover {
        background-color: #218838;
    }

    .delete-btn {
        background-color: #dc3545;
        color: white;
        border: none;
        padding: 6px 12px;
        border-radius: 4px;
        cursor: pointer;
    }

    .delete-btn:hover {
        background-color: #c82333;
    }

    #emptyMessage {
        text-align: center;
        color: #777777;
        font-style: italic;
        margin: 15px 0;
    }
}
```

script3.js

```
// 1. DOM Element Selection
const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Helper function to toggle empty state message
function updateEmptyState() {
  if (taskList.children.length === 0) {
    emptyMessage.style.display = "block";
  } else {
    emptyMessage.style.display = "none";
  }
}

// Function to create and append a new task item
function addTask() {
  const taskText = taskInput.value.trim();

  if (taskText === "") {
    alert("Please enter a task before adding.");
    return;
  }

  // 2. Dynamic Element Creation
  const li = document.createElement("li");
  li.className = "task-item";

  const textSpan = document.createElement("span");
  textSpan.className = "task-text";
  textSpan.textContent = taskText;

  const btnContainer = document.createElement("div");
  btnContainer.className = "btn-group";

  const completeBtn = document.createElement("button");
  completeBtn.className = "complete-btn";
  completeBtn.textContent = "Complete";

  const deleteBtn = document.createElement("button");
  deleteBtn.className = "delete-btn";
  deleteBtn.textContent = "Delete";

  // 3. Event Listeners for Dynamic Elements
  // Toggle task completion
  completeBtn.addEventListener("click", function () {
    textSpan.classList.toggle("completed");
  });

  // Delete task item
  deleteBtn.addEventListener("click", function () {
    li.remove();
    updateEmptyState();
  });

  // 4. Construct the DOM tree
  btnContainer.appendChild(completeBtn);
  btnContainer.appendChild(deleteBtn);

  li.appendChild(textSpan);
  li.appendChild(btnContainer);

  taskList.appendChild(li);

  // Clear input and update empty list message
  taskInput.value = "";
  taskInput.focus();
  updateEmptyState();
}

// 5. Event Listeners
addBtn.addEventListener("click", addTask);

// Allow adding task by pressing the 'Enter' key
taskInput.addEventListener("keydown", function (event) {
  if (event.key === "Enter") {
    addTask();
  }
});
```

});

7. CODE EXPLANATION

Code Element	Description
<code>document.getElementById(...)</code>	Selects the input box, Add Task button, task list, and empty-state message from the DOM.
<code>updateEmptyState()</code>	Checks <code>taskList.children.length</code> and shows or hides the “No tasks available.” message accordingly.
<code>taskInput.value.trim()</code>	Reads the text entered by the user and removes leading/trailing whitespace before validating.
<code>document.createElement("li"/"span"/"div"/"button")</code>	Dynamically creates the task item, its text, its button container, and its Complete/Delete buttons.
<code>li.className / textSpan.className</code>	Assigns CSS classes (<code>task-item</code> , <code>task-text</code>) to newly created elements for styling.
<code>completeBtn.addEventListener("click", ...)</code>	Registers a click handler that toggles the “completed” class on the task text, giving it a strikethrough effect.
<code>textSpan.classList.toggle("completed")</code>	Adds the completed class if absent, or removes it if already present, switching the task's completion status.
<code>deleteBtn.addEventListener("click", ...)</code>	Registers a click handler that removes the task item from the page and refreshes the empty-state message.
<code>li.remove()</code>	Removes the corresponding list item element from the DOM.
<code>appendChild(...)</code>	Builds the task item by nesting the text and buttons inside it, then adds the finished item to the task list.
<code>addBtn.addEventListener("click", addTask)</code>	Runs <code>addTask()</code> whenever the Add Task button is clicked.
<code>taskInput.addEventListener("keydown", ...)</code>	Runs <code>addTask()</code> when the Enter key is pressed while typing in the input box.

8. TEST CASES AND EXECUTION DATA

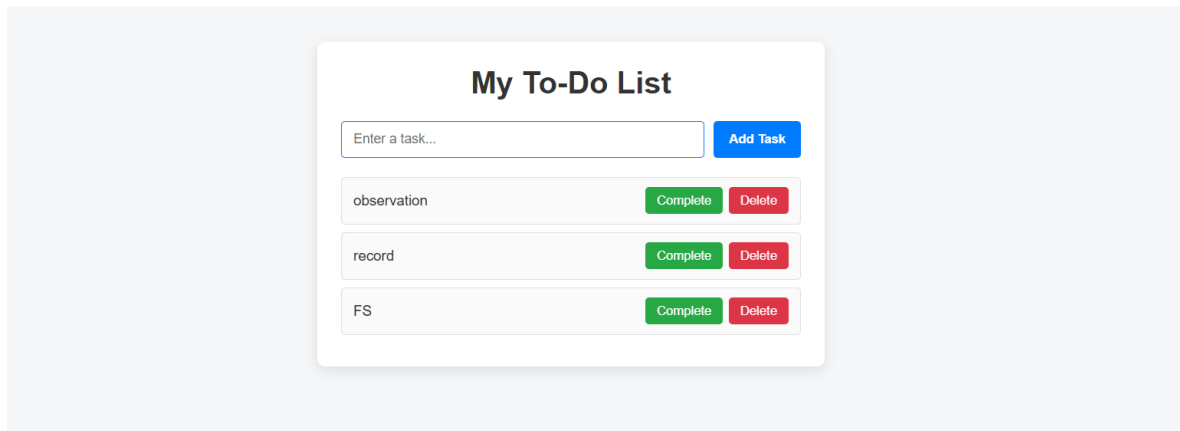
The application was verified in a web browser by adding, completing, and deleting tasks, and checking the displayed output against the expected behaviour.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Page loaded with no tasks added	No tasks available." message is shown	Empty message displayed correctly	Pass
TC-02	Click Add Task with the input box empty	Empty asks the user to enter a task; No task added	No task added	Pass
TC-03	Enter "observation" and click Add Task	task item "observation" appears	Task added and displayed correctly	Pass
TC-04	Add "record" and "FS" as additional tasks	All three tasks are listed in the order added	Added tasks listed correctly	Pass
TC-05	Press Enter after typing a task in the Add Task box	Task added to the list as clicking the Add Task button	Task added correctly via Enter key	Pass
TC-06	Click Complete on a task	Task text shows a strikethrough style	Strikethrough style applied	Pass
TC-07	Click Complete again on the same task	Strikethrough style is removed (toggled)	Style removed correctly	Pass
TC-08	Click Delete on a task	The task is removed from the list	Task removed correctly	Pass
TC-09	Delete all remaining tasks	"No tasks available." message reappears	Empty message reappeared correctly	Pass

9. OUTPUT

Output 1: Task List with Multiple Tasks

Displays the To-Do List after adding three tasks (“observation”, “record”, and “FS”), each with its own Complete and Delete buttons.

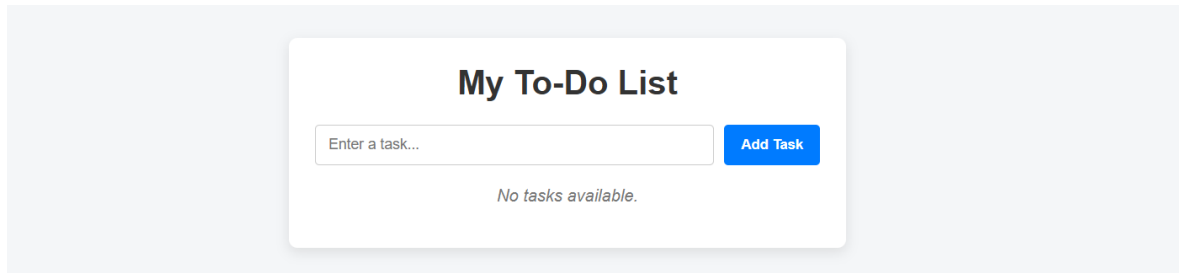


The screenshot displays a web application titled "My To-Do List". At the top, there is a text input field with the placeholder "Enter a task..." and a blue "Add Task" button. Below the input field, there is a list of three tasks, each in a light gray box. The first task is "observation", the second is "record", and the third is "FS". Each task box contains a green "Complete" button and a red "Delete" button.

Task	Complete	Delete
observation	Complete	Delete
record	Complete	Delete
FS	Complete	Delete

Output 2: Empty Task List

Displays the To-Do List in its initial state, with no tasks added, showing the “No tasks available.” message below the input box.



10. RESULT

Thus, an interactive **To-Do List application** was successfully developed using JavaScript, DOM manipulation, and event listeners. Tasks can be added dynamically, marked as completed with a visual strikethrough style, and removed from the list, with an appropriate message shown when the list is empty. The application was verified in a web browser and all features functioned as expected.